

Azure DevOps & Terraform Complete Documentation

Step-by-Step Execution Guide & Complete Source Code Repository (.tf files)

Document Structure & Table of Contents

- **Part 1: Step-by-Step Execution Guide** (Local PowerShell & Azure DevOps CI/CD Pipeline)
- **Part 2: CI/CD & Security Configuration** (azure-pipelines.yml, .tflint.hcl)
- **Part 3: Terraform Environment Configurations** (preprod & prod environments)
- **Part 4: Reusable Infrastructure Modules** (Resource Group, VNet, Subnet, Key Vault, App Gateway, Storage Account, etc.)

Part 1: Step-by-Step Execution Guide

Prerequisites

- **Terraform CLI (v1.15+)** — Verified installed (v1.15.8 on your system)
- **Azure CLI (v2.x+)** — Required for authenticating and managing Azure resources
- **TFLint & Gitleaks (Optional)** — Recommended for static security and compliance scanning

Method 1: Running Locally (PowerShell / Terminal)

Step 1: Login & Select Azure Subscription

```
az login
az account list --output table
az account set --subscription "<YOUR_SUBSCRIPTION_ID_OR_NAME>"
```

Step 2: Format & Validate Code

```
terraform fmt -recursive
cd environments/preprod
terraform init -backend=false
terraform validate
cd ../prod
terraform init -backend=false
terraform validate
cd ../../
```

Step 3: Deploy Preprod Environment

NOTE: Remote backend requires Azure Storage Account storagetechnaccount in Resource Group rg-ravi.

```
cd environments/preprod
terraform init
terraform plan -out=preprod.tfplan
terraform apply preprod.tfplan
```

Step 4: Deploy Production (Prod) Environment

```
cd environments/prod
terraform init
terraform plan -out=prod.tfplan
terraform apply prod.tfplan
```

Step 5: Clean Up Resources (Optional)

```
cd environments/preprod
terraform destroy -auto-approve
cd environments/prod
terraform destroy -auto-approve
```

Method 2: Running via Azure DevOps CI/CD Pipeline

Step 1: Create Azure Remote State Storage Account

```
az group create --name rg-ravi --location eastus
az storage account create --name storagetechnaccount --resource-group rg-ravi --location eastus --sku
Standard_LRS
az storage container create --name tfstate --account-name storagetechnaccount
```

- **Step 2: Service Connection** — Create ARM Service Principal named azure-service-connection in DevOps Project Settings.
- **Step 3: Environments & Approvals** — Create preprod and prod environments in Azure DevOps with manual approval gate on prod.
- **Step 4: Import & Run** — Connect /azure-pipelines.yml to Azure Pipelines and trigger build on PR / push to main.

Pipeline Stage Workflow Summary

Stage	Description	Trigger Condition
1. Validate	Gitleaks secret scan, TFLint analysis, terraform validate	PR / Push to main
2. Preprod Plan	Generates preprod.tfplan execution artifact	After Validate stage
3. Preprod Apply	Applies changes to Preprod environment	Automatic on main branch
4. Prod Plan	Generates prod.tfplan execution artifact	After Preprod Apply stage
5. Prod Apply	Applies changes to Production environment	Requires Manual Approval

Part 2: CI/CD & Security Configuration Files

■ azure-pipelines.yml

```
trigger:
  branches:
    include:
      - main

pr:
  branches:
    include:
      - main

pool:
  vmImage: 'ubuntu-latest'

variables:
  - name: azureServiceConnection
    value: 'azure-service-connection' # Azure DevOps Service Connection name
  - name: terraformVersion
    value: '1.15.8'

stages:
  - stage: Validate
    displayName: 'Validate & Security Scanning'
    jobs:
      - job: ValidateTerraform
        displayName: 'Terraform Format, Validate & Security Scan'
        steps:
          - task: HashiCorpInstaller@1
            displayName: 'Install Terraform'
            inputs:
              terraformVersion: '${terraformVersion}'

          - task: PowerShell@2
            displayName: 'Validate Preprod & Prod with Security Checks'
            inputs:
              targetType: 'inline'
              pwsh: true
              script: |
                Write-Host "===> Running Gitleaks Security Scan for Secrets..."
                curl -sSL https://github.com/gitleaks/gitleaks/releases/download/v8.18.2/gitleaks_8.1
2_linux_x64.tar.gz | tar -xz
                ./gitleaks detect --source . --verbose
```

```

        Write-Host "==> Running TFLint Static Security Analysis..."
        curl -s https://raw.githubusercontent.com/terraform-linters/tflint/master/install_lin
ux.sh | bash

        tflint --init
        tflint

        Write-Host "==> Checking Preprod Infrastructure..."
        Set-Location environments/preprod
        terraform init -backend=false
        terraform fmt -check
        terraform validate

        Write-Host "==> Checking Prod Infrastructure..."
        Set-Location ../prod
        terraform init -backend=false
        terraform fmt -check
        terraform validate

- stage: Preprod_Plan
  displayName: 'Preprod - Plan'

  dependsOn: Validate
  jobs:
    - job: PlanPreprod
      displayName: 'Terraform Plan Preprod'
      steps:
        - task: HashiCorpInstaller@1
          inputs:
            terraformVersion: '${terraformVersion}'

        - task: AzureCLI@2
          displayName: 'Terraform Init & Plan (Preprod)'
          inputs:
            azureSubscription: '${azureServiceConnection}'
            scriptType: 'pscore'
            scriptLocation: 'inlineScript'
            workingDirectory: 'environments/preprod'
            inlineScript: |
              Write-Host "==> Initializing Terraform (Preprod)..."
              terraform init
              Write-Host "==> Planning Terraform Infrastructure (Preprod)..."

              terraform plan -out=preprod.tfplan

- stage: Preprod_Apply
  displayName: 'Preprod - Apply'
  dependsOn: Preprod_Plan
  condition: and(succeeded(), eq(variables['Build.SourceBranch'], 'refs/heads/main'))
  jobs:
    - deployment: ApplyPreprod
      displayName: 'Deploy to Preprod'
      environment: 'preprod'
      strategy:
        runOnce:
          deploy:
            steps:
              - checkout: self
              - task: HashiCorpInstaller@1
                inputs:
                  terraformVersion: '${terraformVersion}'

              - task: AzureCLI@2

```

```

        displayName: 'Terraform Init & Apply (Preprod)'
        inputs:
          azureSubscription: '$(azureServiceConnection)'
          scriptType: 'pscore'
          scriptLocation: 'inlineScript'
          workingDirectory: 'environments/preprod'
          inlineScript: |
            Write-Host "==> Initializing Terraform (Preprod)..."
            terraform init
            Write-Host "==> Applying Terraform Infrastructure (Preprod)..."
            terraform apply -auto-approve

- stage: Prod_Plan
  displayName: 'Prod - Plan'
  dependsOn: Preprod_Apply
  jobs:
    - job: PlanProd
      displayName: 'Terraform Plan Prod'
      steps:
        - task: HashiCorpInstaller@1

          inputs:
            terraformVersion: '$(terraformVersion)'

        - task: AzureCLI@2
          displayName: 'Terraform Init & Plan (Prod)'
          inputs:
            azureSubscription: '$(azureServiceConnection)'
            scriptType: 'pscore'
            scriptLocation: 'inlineScript'
            workingDirectory: 'environments/prod'
            inlineScript: |
              Write-Host "==> Initializing Terraform (Prod)..."
              terraform init
              Write-Host "==> Planning Terraform Infrastructure (Prod)..."
              terraform plan -out=prod.tfplan

- stage: Prod_Apply
  displayName: 'Prod - Apply'
  dependsOn: Prod_Plan
  condition: and(succeeded(), eq(variables['Build.SourceBranch'], 'refs/heads/main'))
  jobs:
    - deployment: ApplyProd
      displayName: 'Deploy to Prod'
      environment: 'prod'
      strategy:
        runOnce:
          deploy:
            steps:
              - checkout: self
              - task: HashiCorpInstaller@1
                inputs:
                  terraformVersion: '$(terraformVersion)'

              - task: AzureCLI@2
                displayName: 'Terraform Init & Apply (Prod)'
                inputs:
                  azureSubscription: '$(azureServiceConnection)'
                  scriptType: 'pscore'
                  scriptLocation: 'inlineScript'
                  workingDirectory: 'environments/prod'

```

```
inlineScript: |  
    Write-Host "==> Initializing Terraform (Prod)..."  
    terraform init  
    Write-Host "==> Applying Terraform Infrastructure (Prod)..."  
    terraform apply -auto-approve
```

■ .tflint.hcl

```
config {  
    format = "compact"  
    plugin_dir = "~/tflint.d/plugins"  
}  
  
plugin "azurerm" {  
    enabled = true  
    version = "0.27.0"  
    source  = "github.com/terraform-linters/tflint-ruleset-azurerm"  
}
```

Part 3: Terraform Environment Configurations

Environment: PREPROD

■ environments\preprod\backend.tf

```
terraform {
  backend "azurerm" {
    resource_group_name = "rg-ravi"
    storage_account_name = "storagetechaccount"
    container_name      = "tfstate"
    key                 = "preprod.terraform.tfstate"
  }
}
```

■ environments\preprod\main.tf

```
data "azurerm_client_config" "current" {}

module "resource_group" {
  source = "../../modules/azurerm_resource_group"
  rgs    = var.rgs
}

module "virtual_network" {
  depends_on = [module.resource_group]
  source     = "../../modules/azurerm_virtual_network"
  vnets     = var.vnets
}

module "subnet" {
  depends_on = [module.virtual_network]
  source     = "../../modules/azurerm_subnet"
  subnets  = var.subnets
}

module "network_security_group" {
  depends_on = [module.resource_group]
  source     = "../../modules/azurerm_network_security_group"
  nsgs      = var.nsgs
}

module "public_ip" {
  depends_on = [module.resource_group]
  source     = "../../modules/azurerm_public_ip"
  public_ips = var.public_ips
}

module "network_interface" {
  depends_on = [module.subnet]
  source     = "../../modules/azurerm_network_interface"
  nics = {
    for k, v in var.nics : k => merge(v, {
      subnet_id = module.subnet.subnets[v.subnet_key].id
    })
  }
}
```

```

module "key_vault" {
  depends_on = [module.resource_group]
  source      = "../../modules/azurerm_key_vault"
  key_vaults = {
    for k, v in var.key_vaults : k => merge(v, {
      tenant_id = data.azurerm_client_config.current.tenant_id
    })
  }
}

module "storage_account" {
  depends_on = [module.resource_group]
  source      = "../../modules/azurerm_storage_account"
  storage_accounts = var.storage_accounts
}

module "application_gateway" {
  depends_on = [module.subnet, module.public_ip]
  source      = "../../modules/azurerm_application_gateway"

  application_gateways = {
    for k, v in var.application_gateways : k => {
      name = v.name
      resource_group_name = v.resource_group_name
      location = v.location
      tags = try(v.tags, {})
      sku = v.sku
      gateway_ip_configuration = {
        name = v.gateway_ip_configuration.name
        subnet_id = module.subnet.subnets[v.gateway_ip_configuration.subnet_key].id
      }
      frontend_ports = v.frontend_ports
      frontend_ip_configurations = [
        for fip in v.frontend_ip_configurations : {
          name = fip.name
          public_ip_address_id = module.public_ip.public_ips[fip.public_ip_key].id
        }
      ]
      backend_address_pools = v.backend_address_pools
      backend_http_settings = v.backend_http_settings

      http_listeners = v.http_listeners
      request_routing_rules = v.request_routing_rules
    }
  }
}

```

■ environments\preprod\variables.tf


```
variable "rgs" {
  type = map(any)
}

variable "vnets" {
  type = map(any)
}

variable "subnets" {
  type = map(any)
}

variable "nsgs" {
  type    = map(any)
  default = {}
}

variable "public_ips" {
  type = map(any)
}

variable "nics" {
  type    = map(any)
  default = {}
}

variable "key_vaults" {
  type    = map(any)
  default = {}
}

variable "storage_accounts" {
  type = map(any)
}

variable "application_gateways" {
  type = map(any)
}
```

■ environments\preprod\terraform.tfvars

```
rgs = {
  rg1 = {
    name      = "rg-chor-dev"
    location  = "centralindia"
  }
  rg2 = {
    name      = "rg-chor-dev-2"
    location  = "centralindia"
  }
}

vnets = {
  vnet1 = {
    name              = "vnet-chor-dev"
    location           = "centralindia"
    resource_group_name = "rg-chor-dev"
    address_space      = ["10.0.0.0/16"]
  }
}
```

```

subnets = {
  snet1 = {
    name                = "agw-subnet"
    resource_group_name = "rg-chor-dev"
    virtual_network_name = "vnet-chor-dev"
    address_prefixes    = ["10.0.1.0/24"]
  },
  snet2 = {
    name                = "backend-subnet"
    resource_group_name = "rg-chor-dev"
    virtual_network_name = "vnet-chor-dev"
    address_prefixes    = ["10.0.2.0/24"]
  }
}

nsgs = {
  nsg1 = {
    name                = "nsg-agw-preprod"
    resource_group_name = "rg-chor-dev"
    location            = "centralindia"

    security_rules = [
      {
        name                = "Allow-HTTP"
        priority            = 100
        direction           = "Inbound"
        access              = "Allow"
        protocol            = "Tcp"
        source_port_range   = "*"
        destination_port_range = "80"
        source_address_prefix = "*"
        destination_address_prefix = "*"
      },
      {
        name                = "Allow-HTTPS"
        priority            = 110
        direction           = "Inbound"
        access              = "Allow"
        protocol            = "Tcp"
        source_port_range   = "*"
        destination_port_range = "443"
        source_address_prefix = "*"
        destination_address_prefix = "*"
      }
    ]
  }
}

public_ips = {
  pip1 = {
    name                = "pip-agw-preprod"
    resource_group_name = "rg-chor-dev"
    location            = "centralindia"
    allocation_method   = "Static"
    sku                 = "Standard"
    tags = {
      environment = "preprod"
    }
  }
}

```

```
nics = {
  nic1 = {
    name                = "nic-web-preprod-01"
    resource_group_name = "rg-chor-dev"
    location             = "centralindia"
    ip_configuration_name = "ipconfig1"
    subnet_key          = "snet2"
    tags = {
      environment = "preprod"
    }
  }
}

key_vaults = {
  kv1 = {
    name                = "kv-chor-preprod-01"
    resource_group_name = "rg-chor-dev"
    location            = "centralindia"
    sku_name            = "standard"
    rbac_authorization_enabled = true

    tags = {
      environment = "preprod"
    }
  }
}

storage_accounts = {
  sa1 = {
    name                = "stchordev01"
    location            = "centralindia"
    resource_group_name = "rg-chor-dev"
    account_tier        = "Standard"
    account_replication_type = "LRS"
  }
}

application_gateways = {
  agw1 = {
    name                = "agw-chor-preprod"
    resource_group_name = "rg-chor-dev"

    location            = "centralindia"
    sku = {
      name      = "Standard_v2"
      tier       = "Standard_v2"
      capacity = 2
    }
    gateway_ip_configuration = {
      name      = "agw-ip-config"
      subnet_key = "snet1"
    }
    frontend_ports = [
      {
        name = "http-port"
        port = 80
      }
    ]
    frontend_ip_configurations = [
      {
        name            = "agw-frontend-ip"
        public_ip_key = "pip1"
      }
    ]
  }
}
```

```
    }
  ]
  backend_address_pools = [
    {
      name          = "backend-pool-dev"
      ip_addresses = ["10.0.2.4", "10.0.2.5"]
    }
  ]
  backend_http_settings = [
    {
      name              = "http-settings"
      cookie_based_affinity = "Disabled"
      port              = 80
      protocol          = "Http"
      request_timeout   = 60
    }
  ]
  http_listeners = [
    {
      name                      = "http-listener"
      frontend_ip_configuration_name = "agw-frontend-ip"
      frontend_port_name         = "http-port"
      protocol                   = "Http"
    }
  ]
  request_routing_rules = [
    {
      name              = "rule-http"
      rule_type         = "Basic"
      http_listener_name = "http-listener"
      backend_address_pool_name = "backend-pool-dev"
      backend_http_settings_name = "http-settings"
      priority          = 100
    }
  ]
  tags = {
    environment = "preprod"
  }
}
```

■ environments\preprod\outputs.tf

```

output "application_gateway_names" {
  description = "Names of deployed Application Gateways"
  value      = { for k, v in module.application_gateway.application_gateways : k => v.name }
}

output "application_gateway_public_ips" {
  description = "Public IP addresses assigned to Application Gateways"
  value      = { for k, v in module.public_ip.public_ips : k => v.ip_address }
}

output "key_vault_uris" {
  description = "URIs of created Key Vaults"
  value      = { for k, v in module.key_vault.key_vaults : k => v.vault_uri }
}

output "network_interface_ids" {
  description = "IDs of created Network Interfaces"
  value      = { for k, v in module.network_interface.nics : k => v.id }
}

```

Environment: PROD

■ environments\prod\backend.tf

```

terraform {
  backend "azurerm" {
    resource_group_name = "rg-ravi"
    storage_account_name = "storagetechnaccount"
    container_name      = "tfstate"
    key                 = "prod.terraform.tfstate"
  }
}

```

■ environments\prod\main.tf

```

data "azurerm_client_config" "current" {}

module "resource_group" {
  source = "../../modules/azurerm_resource_group"
  rgs    = var.rgs
}

module "virtual_network" {
  depends_on = [module.resource_group]
  source     = "../../modules/azurerm_virtual_network"
  vnets     = var.vnets
}

module "subnet" {
  depends_on = [module.virtual_network]
  source     = "../../modules/azurerm_subnet"
  subnets  = var.subnets
}

module "network_security_group" {

```

```

    depends_on = [module.resource_group]
    source      = "../../modules/azurerm_network_security_group"
    nsgs        = var.nsgs
  }

  module "public_ip" {
    depends_on = [module.resource_group]
    source      = "../../modules/azurerm_public_ip"
    public_ips  = var.public_ips
  }

  module "network_interface" {
    depends_on = [module.subnet]
    source      = "../../modules/azurerm_network_interface"
    nics = {
      for k, v in var.nics : k => merge(v, {
        subnet_id = module.subnet.subnets[v.subnet_key].id
      })
    }
  }

  module "key_vault" {
    depends_on = [module.resource_group]
    source      = "../../modules/azurerm_key_vault"
    key_vaults = {
      for k, v in var.key_vaults : k => merge(v, {
        tenant_id = data.azurerm_client_config.current.tenant_id
      })
    }
  }

  module "storage_account" {
    depends_on      = [module.resource_group]
    source          = "../../modules/azurerm_storage_account"
    storage_accounts = var.storage_accounts
  }

  module "application_gateway" {
    depends_on = [module.subnet, module.public_ip]
    source      = "../../modules/azurerm_application_gateway"

    application_gateways = {
      for k, v in var.application_gateways : k => {
        name                = v.name
        resource_group_name = v.resource_group_name
        location            = v.location
        tags                = try(v.tags, {})
        sku                 = v.sku
        gateway_ip_configuration = {
          name                = v.gateway_ip_configuration.name
          subnet_id          = module.subnet.subnets[v.gateway_ip_configuration.subnet_key].id
        }
        frontend_ports      = v.frontend_ports
        frontend_ip_configurations = [
          for fip in v.frontend_ip_configurations : {
            name                = fip.name
            public_ip_address_id = module.public_ip.public_ips[fip.public_ip_key].id
          }
        ]
        backend_address_pools = v.backend_address_pools
        backend_http_settings = v.backend_http_settings
      }
    }
  }

```

```
    http_listeners      = v.http_listeners
    request_routing_rules = v.request_routing_rules
  }
}
```

■ environments\prod\variables.tf

```
variable "rgs" {
  type = map(any)
}

variable "vnets" {
  type = map(any)
}

variable "subnets" {
  type = map(any)
}

variable "nsgs" {
  type    = map(any)
  default = {}
}

variable "public_ips" {
  type = map(any)
}

variable "nics" {
  type    = map(any)
  default = {}
}

variable "key_vaults" {
  type    = map(any)
  default = {}
}

variable "storage_accounts" {
  type = map(any)
}

variable "application_gateways" {
  type = map(any)
}
```

■ environments\prod\terraform.tfvars

```

rgs = {
  rg1 = {
    name      = "rg-chor-prod"
    location  = "centralindia"
  }
}

vnets = {
  vnet1 = {
    name              = "vnet-chor-prod"
    location          = "centralindia"
    resource_group_name = "rg-chor-prod"
    address_space     = ["10.0.0.0/16"]
  }
}

subnets = {
  snet1 = {
    name              = "agw-subnet"
    resource_group_name = "rg-chor-prod"

    virtual_network_name = "vnet-chor-prod"
    address_prefixes     = ["10.0.1.0/24"]
  },
  snet2 = {
    name              = "backend-subnet"
    resource_group_name = "rg-chor-prod"
    virtual_network_name = "vnet-chor-prod"
    address_prefixes     = ["10.0.2.0/24"]
  }
}

nsgs = {
  nsg1 = {
    name              = "nsg-agw-prod"
    resource_group_name = "rg-chor-prod"
    location          = "centralindia"
    security_rules = [
      {
        name              = "Allow-HTTP"
        priority          = 100

        direction          = "Inbound"
        access              = "Allow"
        protocol            = "Tcp"
        source_port_range   = "*"
        destination_port_range = "80"
        source_address_prefix = "*"
        destination_address_prefix = "*"
      },
      {
        name              = "Allow-HTTPS"
        priority          = 110
        direction          = "Inbound"
        access              = "Allow"
        protocol            = "Tcp"
        source_port_range   = "*"
        destination_port_range = "443"
        source_address_prefix = "*"
        destination_address_prefix = "*"
      }
    ]
  }
}

```



```
    }
  }

  public_ips = {
    pip1 = {
      name           = "pip-agw-prod"
      resource_group_name = "rg-chor-prod"
      location        = "centralindia"
      allocation_method = "Static"
      sku             = "Standard"
      tags = {
        environment = "prod"
      }
    }
  }
}

nics = {
  nic1 = {
    name           = "nic-web-prod-01"
    resource_group_name = "rg-chor-prod"

    location        = "centralindia"
    ip_configuration_name = "ipconfig1"
    subnet_key       = "snet2"
    tags = {
      environment = "prod"
    }
  }
}

key_vaults = {
  kv1 = {
    name           = "kv-chor-prod-01"
    resource_group_name = "rg-chor-prod"
    location        = "centralindia"
    sku_name        = "standard"
    rbac_authorization_enabled = true
    tags = {
      environment = "prod"
    }
  }
}

storage_accounts = {
  sa1 = {
    name           = "stchorprod01"
    location        = "centralindia"
    resource_group_name = "rg-chor-prod"
    account_tier     = "Standard"
    account_replication_type = "LRS"
  }
}

application_gateways = {
  agw1 = {
    name           = "agw-chor-prod"
    resource_group_name = "rg-chor-prod"
    location        = "centralindia"
    sku = {
      name = "Standard_v2"
      tier  = "Standard_v2"
    }
  }
}
```

```

    capacity = 2
  }
  gateway_ip_configuration = {
    name      = "agw-ip-config"
    subnet_key = "snet1"
  }
  frontend_ports = [
    {
      name = "http-port"
      port = 80
    }
  ]
  frontend_ip_configurations = [
    {
      name      = "agw-frontend-ip"
      public_ip_key = "pip1"
    }
  ]
  backend_address_pools = [
    {
      name      = "backend-pool-prod"
      ip_addresses = ["10.0.2.10", "10.0.2.11"]
    }
  ]
  backend_http_settings = [
    {
      name              = "http-settings"
      cookie_based_affinity = "Disabled"
      port              = 80
      protocol          = "Http"
      request_timeout   = 60
    }
  ]
  http_listeners = [
    {
      name                      = "http-listener"
      frontend_ip_configuration_name = "agw-frontend-ip"
      frontend_port_name        = "http-port"
      protocol                  = "Http"
    }
  ]
  request_routing_rules = [
    {
      name              = "rule-http"
      rule_type         = "Basic"
      http_listener_name = "http-listener"
      backend_address_pool_name = "backend-pool-prod"
      backend_http_settings_name = "http-settings"
      priority           = 100
    }
  ]
  tags = {
    environment = "prod"
  }
}

```

■ environments\prod\outputs.tf

```
output "application_gateway_names" {
  description = "Names of deployed Application Gateways"
  value       = { for k, v in module.application_gateway.application_gateways : k => v.name }
}

output "application_gateway_public_ips" {
  description = "Public IP addresses assigned to Application Gateways"
  value       = { for k, v in module.public_ip.public_ips : k => v.ip_address }
}

output "key_vault_uris" {
  description = "URIs of created Key Vaults"
  value       = { for k, v in module.key_vault.key_vaults : k => v.vault_uri }
}

output "network_interface_ids" {
  description = "IDs of created Network Interfaces"
  value       = { for k, v in module.network_interface.nics : k => v.id }
}
```

Part 4: Reusable Infrastructure Modules

Module: azurerm_application_gateway

■ modules\azurerm_application_gateway\main.tf

```
resource "azurerm_application_gateway" "appgw" {
  for_each      = var.application_gateways
  name          = each.value.name
  resource_group_name = each.value.resource_group_name
  location      = each.value.location
  tags          = each.value.tags

  sku {
    name      = each.value.sku.name
    tier       = each.value.sku.tier
    capacity  = each.value.sku.capacity
  }

  gateway_ip_configuration {
    name      = each.value.gateway_ip_configuration.name
    subnet_id = each.value.gateway_ip_configuration.subnet_id
  }

  dynamic "frontend_port" {
    for_each = each.value.frontend_ports

    content {
      name = frontend_port.value.name
      port = frontend_port.value.port
    }
  }

  dynamic "frontend_ip_configuration" {
    for_each = each.value.frontend_ip_configurations
    content {
      name                  = frontend_ip_configuration.value.name
      public_ip_address_id = frontend_ip_configuration.value.public_ip_address_id
    }
  }

  dynamic "backend_address_pool" {
    for_each = each.value.backend_address_pools
    content {
      name          = backend_address_pool.value.name
      ip_addresses  = backend_address_pool.value.ip_addresses
      fqdns         = backend_address_pool.value.fqdns
    }
  }
}
```

```

    }
  }

  dynamic "backend_http_settings" {
    for_each = each.value.backend_http_settings
    content {
      name = backend_http_settings.value.name
      cookie_based_affinity = backend_http_settings.value.cookie_based_affinity
      port = backend_http_settings.value.port
      protocol = backend_http_settings.value.protocol
      request_timeout = backend_http_settings.value.request_timeout
    }
  }

  dynamic "http_listener" {
    for_each = each.value.http_listeners
    content {
      name = http_listener.value.name
      frontend_ip_configuration_name = http_listener.value.frontend_ip_configuration_name
      frontend_port_name = http_listener.value.frontend_port_name

      protocol = http_listener.value.protocol
    }
  }

  dynamic "request_routing_rule" {
    for_each = each.value.request_routing_rules
    content {
      name = request_routing_rule.value.name
      rule_type = request_routing_rule.value.rule_type
      http_listener_name = request_routing_rule.value.http_listener_name
      backend_address_pool_name = request_routing_rule.value.backend_address_pool_name
      backend_http_settings_name = request_routing_rule.value.backend_http_settings_name
      priority = request_routing_rule.value.priority
    }
  }
}

```

■ modules\azurerm_application_gateway\variables.tf

```

variable "application_gateways" {
  description = "Map of Application Gateways to create"
  type = map(object({
    name = string
    resource_group_name = string
    location = string
    sku = object({
      name = string
      tier = string
      capacity = optional(number, 2)
    })
    gateway_ip_configuration = object({
      name = string
      subnet_id = string
    })
    frontend_ports = list(object({
      name = string
      port = number
    }))
    frontend_ip_configurations = list(object({

```

```

        name                = string
        public_ip_address_id = string
    )))
    backend_address_pools = list(object({
        name                = string
        ip_addresses        = optional(list(string))
        fqdns                = optional(list(string))
    }))
    backend_http_settings = list(object({
        name                = string
        cookie_based_affinity = string
        port                = number
        protocol            = string
        request_timeout      = number
    }))
    http_listeners = list(object({
        name                = string
        frontend_ip_configuration_name = string
        frontend_port_name    = string
        protocol              = string
    }))
    request_routing_rules = list(object({
        name                = string
        rule_type           = string
        http_listener_name  = string
        backend_address_pool_name = string
        backend_http_settings_name = string
        priority            = number
    }))
    tags = optional(map(string), {})
  })))
}

```

■ modules\azurerm_application_gateway\outputs.tf

```

output "application_gateways" {
  description = "Map of created Application Gateways"
  value       = azurerm_application_gateway.appgw
}

```

Module: azurerm_key_vault

■ modules\azurerm_key_vault\main.tf

```

resource "azurerm_key_vault" "key_vaults" {
  for_each      = var.key_vaults
  name          = each.value.name
  location      = each.value.location
  resource_group_name = each.value.resource_group_name
  tenant_id     = each.value.tenant_id
  sku_name      = each.value.sku_name
  soft_delete_retention_days = each.value.soft_delete_retention_days
  purge_protection_enabled = each.value.purge_protection_enabled
  rbac_authorization_enabled = try(each.value.rbac_authorization_enabled, try(each.value.enable_rbac_
authorization, true))
  tags          = each.value.tags
}

```

■ modules\azurerm_key_vault\variables.tf

```

variable "key_vaults" {
  description = "Map of Key Vaults to create"
  type = map(object({
    name                = string
    resource_group_name = string
    location            = string
    sku_name            = optional(string, "standard")
    tenant_id           = string
    soft_delete_retention_days = optional(number, 7)
    purge_protection_enabled = optional(bool, false)
    rbac_authorization_enabled = optional(bool, true)
    tags                = optional(map(string), {})
  }))
}

```

■ modules\azurerm_key_vault\outputs.tf

```

output "key_vaults" {
  description = "Map of created Key Vaults"
  value      = azurerm_key_vault.key_vaults
}

```

Module: azurerm_network_interface

■ modules\azurerm_network_interface\main.tf

```

resource "azurerm_network_interface" "nics" {
  for_each      = var.nics
  name          = each.value.name
  location      = each.value.location
  resource_group_name = each.value.resource_group_name
  tags         = each.value.tags

  ip_configuration {
    name                = each.value.ip_configuration_name
    subnet_id           = each.value.subnet_id
    private_ip_address_allocation = each.value.private_ip_address_allocation
    private_ip_address   = each.value.private_ip_address
  }
}

```

■ modules\azurerm_network_interface\variables.tf

```

variable "nics" {
  description = "Map of Network Interfaces to create"
  type = map(object({
    name                = string
    resource_group_name = string
    location            = string
    ip_configuration_name = string
    subnet_id           = string
    private_ip_address_allocation = optional(string, "Dynamic")
    private_ip_address   = optional(string)
    tags                = optional(map(string), {})
  }))
}

```

■ modules\azurerm_network_interface\outputs.tf

```

output "nics" {
  description = "Map of created Network Interfaces"
  value      = azurerm_network_interface.nics
}

```

Module: azurerm_network_security_group

■ modules\azurerm_network_security_group\main.tf

```

resource "azurerm_network_security_group" "nsgs" {
  for_each      = var.nsgs
  name          = each.value.name
  location      = each.value.location
  resource_group_name = each.value.resource_group_name
  tags          = each.value.tags

  dynamic "security_rule" {
    for_each = each.value.security_rules
    content {
      name                = security_rule.value.name
      priority            = security_rule.value.priority
      direction           = security_rule.value.direction
      access              = security_rule.value.access
      protocol            = security_rule.value.protocol
      source_port_range   = security_rule.value.source_port_range
      destination_port_range = security_rule.value.destination_port_range
      source_address_prefix = security_rule.value.source_address_prefix
      destination_address_prefix = security_rule.value.destination_address_prefix
    }
  }
}

```

■ modules\azurerm_network_security_group\variables.tf

```

variable "nsgs" {
  description = "Map of Network Security Groups and security rules to create"
  type = map(object({
    name                = string
    resource_group_name = string
    location            = string
    security_rules = optional(list(object({
      name                = string
      priority            = number
      direction           = string
      access              = string
      protocol            = string
      source_port_range   = string
      destination_port_range = string
      source_address_prefix = string
      destination_address_prefix = string
    })), [])
    tags = optional(map(string), {})
  }))
}

```

■ modules\azurerm_network_security_group\outputs.tf


```
output "nsgs" {
  description = "Map of created Network Security Groups"
  value      = azurerm_network_security_group.nsgs
}
```

Module: azurerm_public_ip

■ modules\azurerm_public_ip\main.tf

```
resource "azurerm_public_ip" "public_ips" {
  for_each      = var.public_ips
  name         = each.value.name
  resource_group_name = each.value.resource_group_name
  location     = each.value.location
  allocation_method = each.value.allocation_method
  sku          = each.value.sku
  tags         = each.value.tags
}
```

■ modules\azurerm_public_ip\variables.tf

```
variable "public_ips" {
  description = "Map of public IPs to create"
  type = map(object({
    name           = string
    resource_group_name = string
    location       = string
    allocation_method = string
    sku            = optional(string, "Standard")
    tags           = optional(map(string), {})
  }))
}
```

■ modules\azurerm_public_ip\outputs.tf

```
output "public_ips" {
  description = "Map of created public IPs"
  value      = azurerm_public_ip.public_ips
}
```

Module: azurerm_resource_group

■ modules\azurerm_resource_group\main.tf

```
resource "azurerm_resource_group" "resource_group" {
  for_each = var.rgs
  name     = each.value.name
  location = each.value.location
  tags     = each.value.tags
}
```

■ modules\azurerm_resource_group\variables.tf

```
variable "rgs" {
  description = "Map of resource groups to create"
  type = map(object({
    name      = string
    location  = string
    tags      = optional(map(string), {})
  }))
}
```

■ modules\azurerm_resource_group\outputs.tf

```
output "resource_groups" {
  description = "Map of created resource groups"
  value       = azurerm_resource_group.resource_group
}
```

Module: azurerm_storage_account

■ modules\azurerm_storage_account\main.tf

```
resource "azurerm_storage_account" "storage_accounts" {
  for_each      = var.storage_accounts
  name          = each.value.name
  resource_group_name = each.value.resource_group_name
  location      = each.value.location
  account_tier  = each.value.account_tier
  account_replication_type = each.value.account_replication_type
  tags          = each.value.tags
}
```

■ modules\azurerm_storage_account\variables.tf

```
variable "storage_accounts" {
  description = "Map of storage accounts to create"
  type = map(object({
    name              = string
    resource_group_name = string
    location          = string
    account_tier      = string
    account_replication_type = string
    tags              = optional(map(string), {})
  }))
}
```

■ modules\azurerm_storage_account\outputs.tf

```
output "storage_accounts" {
  description = "Map of created storage accounts"
  value       = azurerm_storage_account.storage_accounts
}
```

Module: azurerm_subnet

■ modules\azurerm_subnet\main.tf

```
resource "azurerm_subnet" "subnets" {
  for_each      = var.subnets
  name          = each.value.name
  resource_group_name = each.value.resource_group_name
  virtual_network_name = each.value.virtual_network_name
  address_prefixes = each.value.address_prefixes
}
```

■ modules\azurerm_subnet\variables.tf

```
variable "subnets" {
  description = "Map of subnets to create"
  type = map(object({
    name          = string
    resource_group_name = string
    virtual_network_name = string
    address_prefixes = list(string)
  }))
}
```

■ modules\azurerm_subnet\outputs.tf

```
output "subnets" {
  description = "Map of created subnets"
  value      = azurerm_subnet.subnets
}
```

Module: azurerm_virtual_network

■ modules\azurerm_virtual_network\main.tf

```
resource "azurerm_virtual_network" "vnets" {
  for_each      = var.vnets
  name          = each.value.name
  location      = each.value.location
  resource_group_name = each.value.resource_group_name
  address_space = each.value.address_space
  tags          = each.value.tags
}
```

■ modules\azurerm_virtual_network\variables.tf

```
variable "vnets" {
  description = "Map of virtual networks to create"
  type = map(object({
    name          = string
    location      = string
    resource_group_name = string
    address_space = list(string)
    tags          = optional(map(string), {})
  }))
}
```

■ modules\azurerm_virtual_network\outputs.tf

```
output "vnets" {
  description = "Map of created virtual networks"
  value      = azurerm_virtual_network.vnets
}
```

